

Requirement Specification:

Enable user GitLab & Docker images

Amit Karpe, Lead Engineer

31 Jan 2025

Contents

0.0.1	1. INTRODUCTION	5
0.0.2	2. APPLICATION OVERVIEW	5
0.0.3	3. ARCHITECTURALLY SIGNIFICANT REQUIREMENTS	6
0.0.4	4. SECURITY	6
0.0.5	5. COMMON MODULES	6
0.0.6	6. FUNCTIONAL MODULES	7
0.0.7	7. MESSAGE / SERVICE INTERFACE	7
0.0.8	8. DATABASE DESIGN	7
0.0.9	9. APPENDIX A	7

Prepared By

Name	Project Role	Signature	Date
Amit Karpe	Lead Engineer		31 Jan 2025

Reviewed By

Name	Project Role	Signature	Date
Yeo Zhen Xuan	Tech Lead		31 Jan 2025

Approved By

Name	Project Role	Signature	Date
Luke Lim	Synapxe Project Manager		31 Jan 2025
Kevin Lam	TRUST Business Lead		31 Jan 2025

Table of Contents

1. INTRODUCTION

- 1.1 References

2. APPLICATION OVERVIEW

- 2.1 Layers or architectural framework
- 2.2 Deployment View
- 2.3 Development View
- 2.4 Architectural Mechanisms

3. ARCHITECTURALLY SIGNIFICANT REQUIREMENTS

4. SECURITY

- 4.1 System Security
- 4.2 Data Security
- 4.3 Application Security

5. COMMON MODULES

- 5.1
 - 5.1.1 Description
 - 5.1.2 Design Model
 - 5.1.3 Special Notes or Consideration
 - 5.1.4 Usage Example

6. FUNCTIONAL MODULES

- 6.1
 - 6.1.1 Description
 - 6.1.2 Design Model
 - 6.1.3 Special Notes or Consideration

7. MESSAGE / SERVICE INTERFACE

- 7.1 Overview
- 7.2 Inputs
 - 7.2.1 <Interface / Service Name>
- 7.3 Outputs
 - 7.3.1 <Interface / Service Name>

8. DATABASE DESIGN

- 8.1 Entity Relationship Diagram
- 8.2 Data Dictionary
- 8.3 Stored Procedures / Functions

9. APPENDIX A

0.0.1 1. INTRODUCTION

1.1 Document Purpose This document defines the design specifications for deploying code, container repositories, and related cloud infrastructure in a non-internet environment within Singapore Government on Commercial Cloud (GCC) by GovTech.

1.2 System Overview Tech Deliverable

4a. Enable Lifebit users to run their own codes

- Phase 1 – Setup repositories (GitLab and AWS ECR) for hosting, accessing & managing private Docker tools & scripts/codes
- Phase 2 – Setup to support hosting, accessing & managing NextFlow pipeline

Problem Statement

Currently, there is no version control repository available within the TRUST-Lifebit platform where researchers can host their custom codes and pipelines.

Outcome

CloudOS user can securely access and perform version control and collaborate with fellow researchers of their private codes, docker tools, and pipelines using Gitlab account and docker images using AWS ECR.

1.3 Project Objectives / Goals

- Enable private code and container repository hosting within a closed cloud environment.
- Provide a scalable and secure solution for pipeline execution and container management.
- Ensure compliance with regulatory and security requirements.

1.4 Definitions/Glossary

- **GitLab Server:** Self-hosted version of GitLab for code repository management.
- **AWS ECR:** Private container image repository for storing and managing Docker images.
- **AWS Batch:** Managed batch processing service for running containerized workloads.
- **CloudOS:** Lifebit's cloud-native bioinformatics platform.

1.5 References

- GovTech GCC Documentation
- AWS Security Best Practices

0.0.2 2. APPLICATION OVERVIEW

2.1 Layers or Architectural Framework The solution follows a multi-account AWS architecture for hosting CloudOS and related infrastructure components:

- **AWS Service Account:** Hosts CloudOS and GitLab Server for private code repositories.
- **AWS Workload Account:** Hosts AWS Batch and AWS ECR for containerized workload execution.

2.2 Deployment View

- **GitLab Server:** Deployed in the AWS Service Account, configured for private access.
- **AWS ECR:** Deployed in the AWS Workload Account, accessible only within AWS GCC accounts.
- **AWS Batch:** Utilized for running containerized jobs, with required IAM permissions.

2.3 Development View

- GitLab provides private repository hosting for researchers and engineers.
- CI/CD workflows will be managed within GitLab to integrate with AWS Batch.
- AWS ECR will store and manage private container images for workloads.

2.4 Architectural Mechanisms

- Secure networking between AWS accounts using VPC peering or AWS PrivateLink.
- IAM roles and policies restricting access to only necessary resources.

0.0.3 3. ARCHITECTURALLY SIGNIFICANT REQUIREMENTS

Deployment in a non-internet environment within Singapore GCC.

Secure and private access for all services.

Multi-account strategy for isolation and compliance.

0.0.4 4. SECURITY

4.1 System Security

- Network access controls enforced via Security Groups and VPC.
- AWS KMS for encrypting sensitive data and images.

4.2 Data Security

- No external data access permitted; all data resides within AWS GCC.
- Private container images stored in AWS ECR with strict access policies.

4.3 Application Security

- GitLab access restricted to authorized users only.
- IAM roles configured for least privilege access.

0.0.5 5. COMMON MODULES

5.1

5.1.1 Description NA

5.1.2 Design Model NA

5.1.3 Special Notes or Consideration NA

5.1.4 Usage Example NA

0.0.6 6. FUNCTIONAL MODULES

6.1

6.1.1 Description NA

6.1.2 Design Model NA

6.1.3 Special Notes or Consideration NA

0.0.7 7. MESSAGE / SERVICE INTERFACE

7.1 Overview NA

7.2 Inputs

7.2.1 <Interface / Service Name> NA

7.3 Outputs

7.3.1 <Interface / Service Name> NA

0.0.8 8. DATABASE DESIGN

8.1 Entity Relationship Diagram NA

8.2 Data Dictionary NA

8.3 Stored Procedures / Functions NA

0.0.9 9. APPENDIX A

NA
